

Krishna Mula

+1 (408) 569-6676 | krishna.mula@sjsu.edu | github.com/krishmula | krishmula.github.io | linkedin.com/krishna-mula | San Jose, CA

EDUCATION

San Jose State University

Master of Science in Computer Science

Coursework: Distributed Systems, Advanced Parallel Processing, Cloud Computing, Database Systems, Machine Learning.

San Jose, CA

Aug 2025 - May 2027

EXPERIENCE

Intake AI

Software Engineer Intern

San Francisco, CA

Jun 2026 - Present

- Shipped a **GPU cross-encoder reranking service** on ECS-on-EC2 (NVIDIA L4, FastAPI, fp16) serving every chat request, with readiness gated by a warm-up inference and a timeout, bulkhead, and circuit breaker that degrade to fusion ranking under load.
- Cut reranking **p50 latency 33%** (986 to 662 ms) and **peak GPU memory 46%** by diagnosing a batch size shipped 8x too large; sized the concurrency budget from measured p99 so concurrent requests queue instead of silently skipping the model.
- Built the **evaluation harness** the team's quality claims run on: quote-anchored golden set with a locked holdout, production functions driven directly, and LLM judges calibrated against blind human labels; used it to select the production reranker from a four-model matrix and to prove the latency wins cost no ranking quality.
- Shrank the GPU worker container from **11.8 GB to 908 MB** and cut its memory reservation 20x after measuring peak usage at 395 MiB against an 8 GB request, verified against a live production-shape run.
- Rebuilt document extraction from vision-model transcription to a **deterministic PyMuPDF parse layer** anchoring every extracted claim to a character span, making hallucination detectable by construction.
- Migrated production from a single EC2 host to a **two-account, event-driven AWS deployment**: ECS Fargate with SQS-backed workers, Spot capacity with backlog-driven autoscaling, and Terraform modules with per-environment roots.

PwC

Software Engineer

Aug 2022 - Jul 2025

- Architected a **RAG pipeline** for IT security compliance, embedding documents into ChromaDB for semantic retrieval and integrating **LLM inference** for automated validation against regulatory standards, **reducing review time by 70%**.
- Designed a **RabbitMQ-backed async ingestion architecture** that decoupled upload APIs from parsing, chunking, embedding, and vector indexing, **reducing latency by 30%** under bursty workloads.
- Built an ETL pipeline using PyRFC to extract SM20/SM21 security audit logs from **5 production SAP systems**, replacing a process that required **2+ hours of manual export and formatting** per report.
- Automated security controls execution on client SAP systems by codifying **25+ controls** as Flask + MongoDB REST APIs, ingesting data directly from SAP HANA and validating against security specifications.
- Deployed serverless REST APIs on Azure Functions and AWS Lambda with pre-warm triggers, **serving compliance reports across 15+ systems**, optimizing for 20-minute twice-daily burst windows.

Software Engineering Intern

Feb 2022 - Aug 2022

- Built Nivo.js dashboards tracking security control pass/fail rates, compliance coverage, and historical trends, letting teams prioritize which controls to automate next.
- Automated deployments with **Azure CI/CD** pipelines, reducing release cycles from **2 days to 4.5 hours**.

PROJECTS

Aether | *Python · TCP Sockets · FastAPI · Docker SDK · OpenTelemetry · Prometheus · D3.js*

- Built a **distributed pub-sub messaging system** over raw TCP with a gossip-based multi-broker mesh, UUID deduplication, configurable fanout/TTL, and heartbeat-driven peer eviction.
- Implemented **Chandy-Lamport global snapshots** with replication to k peers driving hybrid broker failover; a fresh snapshot triggers full state recovery and a stale one redistributes subscribers to least-loaded brokers.
- Built a **FastAPI orchestration control plane** over the Docker SDK with a pull-based subscriber assignment registry as the source of truth during recovery; subscribers detect failure via Ping/Pong and self-heal without coordinated restarts.

TECHNICAL SKILLS

Languages: Python, TypeScript, SQL, C++.

ML & Inference: PyTorch, Hugging Face Transformers, cross-encoder reranking, fp16 GPU serving, ChromaDB.

Frameworks: FastAPI, Flask, Node.js, React, Next.js.

Infrastructure: Terraform, Docker, AWS (ECS, Fargate, EC2, ECR, Lambda, SQS, S3, RDS, DynamoDB, CloudWatch, IAM/OIDC), Azure (Functions, VM, Entra ID), GitHub Actions, Nginx.

Data & Messaging: PostgreSQL, MongoDB, Redis, Cassandra, SQLite, RabbitMQ, SQS.

Observability: OpenTelemetry, Prometheus, Grafana, Loki, Sentry.